НУ «ЛЬВІВСЬКА ПОЛІТЕХНІКА» · КАФЕДРА СИСТЕМ ШТУЧНОГО ІНТЕЛЕКТУ

ПАМ'ЯТКА ТА ШПАРГАЛКА ДО ЗАХИСТУ ЛАБОРАТОРНОЇ РОБОТИ №6

Тема: Адаптивна компонентна верстка інтерфейсу вебсайту за допомогою Tailwind CSS

Студент: Лемішка Андрій Андрійович (група ПП-24) · Проєкт: ChargeUA

Живий деплой: https://mellifluous-eclair-8fe561.netlify.app/ · GitHub: AndriyLemishka/SecondYear

1. ВСТУПНИЙ СПІЧ НА ЗАХИСТІ (ЯК ПРЕЗЕНТУВАТИ ЗА 90 СЕКУНД)

🎤 **ВАШ ГОТОВИЙ СПІЧ:** «Доброго дня! У Лабораторній роботі №6 я виконав рефакторинг та розробку адаптивних компонентів для національного агрегатора зарядних станцій ChargeUA з використанням Tailwind CSS.
1. Основна концепція: ChargeUA — це інформаційний хаб і калькулятор (ми не проводимо транзакції і не бронюємо, а допомагаємо водію електромобіля знайти швидкісний порт і розрахувати вартість сесії).
2. Архітектура: застосовано гібридний підхід. Ми зберегли дизайн-токени (CSS Custom Properties у :root) і поєднали їх з utility-first класами Tailwind CSS (container, flex, grid, rounded-xl, sm:flex-row).
3. Mobile-First: реалізовано адаптивну навігацію з доступним гамбургер-меню (WCAG 2.2, керування через клавіатуру, Escape, блокування фонового скролу), гнучкі сітки для 375px (мобільні), 768px (планшети) та 1440px (десктоп).
4. Продуктивність та оптимізація: карту Leaflet та калькулятор переведено на ліниве завантаження через IntersectionObserver, а стилі Tailwind скомпільовано в tailwind.min.css. Завдяки цьому в аудиті Google Lighthouse ми досягли: Best Practices — 100%, SEO — 100%, Accessibility — 97%, Performance — 87% з ідеальним Total Blocking Time 0 мс та CLS 0.
Проєкт повністю валідний за стандартом W3C (0 помилок) і опублікований на Netlify. Готовий продемонструвати функціонал і код!»

2. ТОП-8 ПИТАНЬ ВИКЛАДАЧА ТА ВПЕВНЕНІ ВІДПОВІДІ (БЛІЦ-ОПИТУВАННЯ)

❓ 1. Чим Tailwind CSS відрізняється від компонентних фреймворків на зразок Bootstrap?

🎯 **Відповідь:** Bootstrap дає готові монолітні компоненти (.btn, .card, .modal) з наперед визначеним дизайном, через що сайти виглядають однотипно, а перевизначення стилів потребує громіздкого CSS. Tailwind CSS — це utility-first фреймворк, що надає атомарні низькорівневі утиліти (flex, p-4, bg-blue-600, rounded-xl). Це дає абсолютну свободу створення унікального дизайну без написання кастомного CSS-коду. Крім того, завдяки JIT-компіляції у фінальний бандл потрапляють лише реально використані класи, тому результуючий CSS важить лічені кілобайти.

💻 **Що показати / ключовий код:** `index.html: class='card p-4 rounded-xl shadow-sm bg-white border border-slate-200'`

❓ 2. Як у проєкті поєднано власний style.css та Tailwind CSS?

🎯 **Відповідь:** Ми використали гібридну архітектуру. У style.css збережено глобальні дизайн-токени бренду ChargeUA (:root з фірмовими кольорами, розмірами шрифтів clamp(), відступами та базовим скиданням CSS Reset). Натомість для сіток, карток станцій, адаптивних лейаутів та інтерактивних станів використано Tailwind CSS. Це забезпечує стабільність дизайн-системи та швидкість розробки компонентів.

💻 **Що показати / ключовий код:** `style.css (:root { --color-primary: #005EC9; }) + tailwind.min.css (утиліти)`

❓ 3. Як реалізовано принцип Mobile-First у Tailwind та чистому CSS?

🎯 **Відповідь:** Принцип Mobile-First означає, що базові стилі (без префіксів) пишуться для найменших мобільних екранів (320–375px), а для ширших екранів стилі послідовно розширюються через медіа-запити min-width. У Tailwind це працює автоматично: класи без префікса (наприклад, flex flex-col) діють на мобільних, а префікси sm: (від 640px) та md: (від 768px) активуються лише при розширенні вікна браузера. Усі сенсорні зони (кнопки, чипси) мають мінімум 44×44px для зручного натискання пальцем.

💻 **Що показати / ключовий код:** `class='flex flex-col sm:flex-row md:items-end gap-3' (1 колонка на мобільних -> горизонтально від 640px)`

❓ 4. Як працює гамбургер-меню з точки зору веб-доступності (WCAG 2.2 AA)?

🎯 **Відповідь:** Модуль js/nav.js забезпечує повну доступність для клавіатури та скрінрідерів: кнопка має семантичний тег <button>, атрибути aria-label='Відкрити меню навігації', aria-controls='site-nav' та динамічний aria-expanded='true/false'. При відкритті меню на мобільному скрол сторінки блокується (overflow: hidden), фокус залишається доступним, а закриття реалізовано трьома способами: кліком на кнопку-хрестик, натисканням клавіші Escape, або кліком на будь-яке посилання навігації.

💻 **Що показати / ключовий код:** `js/nav.js: burgerBtn.setAttribute('aria-expanded', isExpanded); document.addEventListener('keydown', (e) => e.key === 'Escape' && closeNav())`

❓ 5. Як влаштований інтерактивний калькулятор зарядки ChargeUA?

🎯 **Відповідь:** Калькулятор реалізовано на чистому JavaScript (js/calculator.js). Він містить базу даних понад 45 моделей електромобілів (Tesla, VW, Nissan, BMW, BYD) з реальними параметрами: ємність батареї, максимальна потужність DC, тип порту та розхід енергії. Користувач обирає авто (або вводить кВт·год вручну) і рухає інтерактивний повзунок від 20% до 80%. Скрипт динамічно розраховує: додані кВт·год, вартість за тарифом 14.12 ₴/кВт·год, час зарядки на станції 120 кВт та економію проти бензину А-95.

💻 **Що показати / ключовий код:** `js/calculator.js: sessionCost = addedKwh * car.tariff; savings = benzinCost - sessionCost`

❓ 6. Як реалізовано ліниве завантаження карти (Leaflet) та чому це важливо?

🎯 **Відповідь:** Інтерактивна карта Leaflet та тайли Esri ArcGIS важать понад 280 КБ і вимагають десятків мережевих HTTP-запитів. Щоб карта не гальмувала перший рендеринг сторінки, ми використали IntersectionObserver: сам скрипт leaflet.js і ініціалізація карти запускаються лише тоді, коли блок #map реально перетинає вікно перегляду (rootMargin: '0px'). Під час початкового завантаження сторінки час навантаження процесора (CPU) дорівнює 0 мс.

💻 **Що показати / ключовий код:** `js/map.js: observer = new IntersectionObserver((entries) => { if (entries[0].isIntersecting) loadLeaflet(initMap); })`

❓ 7. Які результати аудитів W3C та Google Lighthouse?

🎯 **Відповідь:** 1. W3C Nu HTML Checker: перевірка сторінки index.html показала 0 помилок і 0 попереджень (повна валідність HTML5).
2. Google Lighthouse: на опублікованому на Netlify сайті досягнуто: Best Practices — 100/100, SEO — 100/100, Accessibility — 97/100 (зразковий рівень WCAG) та Performance — 87/100 з Total Blocking Time (TBT) рівно 0 мс та зсувом макета (CLS) рівно 0.

💻 **Що показати / ключовий код:** `Звіт Lighthouse зафіксовано на Рис. 15 у розділі 4 звіту.`

❓ 8. Чому в проєкті ChargeUA немає бронювання та оплати онлайн?

🎯 **Відповідь:** Згідно з технічним завданням, ChargeUA є незалежним національним агрегатором станцій різних операторів (YASNO, AutoEnterprise, TOKA, EcoFactor). Агрегатор надає інформаційні послуги: пошук вільних портів у реальному часі, фільтрацію за конекторами, карту покриття та розрахунок вартості. Він свідомо не проводить транзакцій і не бере комісій, направляючи водія до відповідного застосунку оператора.

💻 **Що показати / ключовий код:** `Розділ 'Юридичні аспекти агрегатора' у звіті.`

3. ПОКРОКОВИЙ ПЛАН ДЕМОНСТРАЦІЇ (ЩО ПОКАЗАТИ ВИКЛАДАЧУ ЗА 60 СЕКУНД)

Дія / Етап показу	Що саме коментувати викладачу
Крок 1. Відкрити живий сайт у браузері	https://mellifluous-eclair-8fe561.netlify.app/ — показати швидке завантаження, адаптивний хедер, логотип та чіткий геро-банер.
Крок 2. Продемонструвати калькулятор зарядки	Натиснути на швидкий пресет авто (наприклад, Tesla або VW ID.4), порухати повзунок відсотка заряду — показати, як миттєво змінюється ціна, час сесії та економія.
Крок 3. Натиснути 'Знайти станцію'	Клік на кнопку плавно скролить сторінку до карти та автоматично активує фільтр конектора під обране авто (наприклад, CCS 2).
Крок 4. Показати інтерактивну карту	Показати маркери станцій (Львів, Київ, траси), клікнути на маркер — відкривається поп-ап з живою кількістю вільних портів і тарифом.
Крок 5. Фільтрація каталогу станцій	Клікнути на фільтр міст (Львів, Київ, Одеса) або кнопку 'Тільки вільні' — показати миттєве оновлення карток та кнопку 'Показати ще'.
Крок 6. DevTools — режим смартфона (375px)	Натиснути F12 -> іконка смартфона (375px): показати роботу гамбургер-меню (відкриття/закриття/Escape) та 1-колонковий адаптивний лейаут.
Крок 7. Показати код у VS Code	Відкрити index.html (показати класи Tailwind, семантичні теги header/main/section/footer), style.css (:root токени) та скрипти у js/.

4. ПІДСТУПНІ ЗАПИТАННЯ ВИКЛАДАЧА («TRICKY QUESTIONS»)

❓ Т1. «А якщо у клієнта старий браузер або вимкнений JavaScript, чи сайт зламається?»

🎯 **Відповідь:** Ні, сайт спроєктовано за принципом прогресивного покращення (Progressive Enhancement). Вся базова розмітка, текстовий контент, характеристики станцій, каталог та початковий розрахунок калькулятора статично згенеровані безпосередньо в HTML. Якщо JS вимкнено, користувач все одно бачить повноцінний сайт, контакти та станції. JavaScript лише додає динамічну інтерактивність: перерахунок формул і фільтрацію списку.

❓ Т2. «Чому в коді одночасно є і tailwind.min.css, і cdn.tailwindcss.com?»

🎯 **Відповідь:** tailwind.min.css — це заздалегідь скомпільований файл стилів (11 КБ), який завантажується миттєво і усуває затримку рендерингу (FCP 0.6 с). Офіційний скрипт CDN ми підключили згідно з обов'язковою вимогою Етапу 3

лабораторної роботи. Але щоб CDN-парсер не блокував перший показ сторінки на мобільних пристроях, він підключається неблокуюче, у фоні.

❓ ТЗ. «Чому ви не використали React, Vue або Angular замість ванільного JS?»

🎯 **Відповідь:** Для лендінгу та інформаційного агрегатора тягнути важкі SPA-фреймворки (150–300 КБ бандлу) — це надлишковий оверхед (overengineering), який критично погіршує швидкість завантаження (Core Web Vitals). Чистий ванільний JavaScript дозволив реалізувати весь функціонал калькулятора, фільтрів та меню у легких файлах загальним обсягом менше 20 КБ і забезпечити 0 мс блокування процесора.